Sistema multiagente multicast

Avaliação didática de risco cardiológico com LPA2v

RELATÓRIO DE ARQUITETURA | Java SE 17+ | 19 de outubro de 2026

Cinco processos independentes cooperam por um único grupo multicast. Quatro agentes transformam respostas e medidas em evidências. O coordenador correlaciona as mensagens, controla o prazo e consolida três nós de análise paraconsistente.

Entrega preparada	Verificação disponível
Projeto Java completo	5 processos, multicast exclusivo, sem frameworks
Cálculos e validação	210 verificações unitárias aprovadas
Comunicação real local	6 casos e cenários de falha/concorrência aprovados
Demonstração distribuída	Pendente: laboratório, três máquinas e capturas reais

Este documento descreve o código presente em src/br/edu/cardio e os testes reais registrados em evidencias/local. A implementação usa apenas a API padrão do Java SE; Python e ReportLab são ferramentas opcionais para a produção deste PDF.

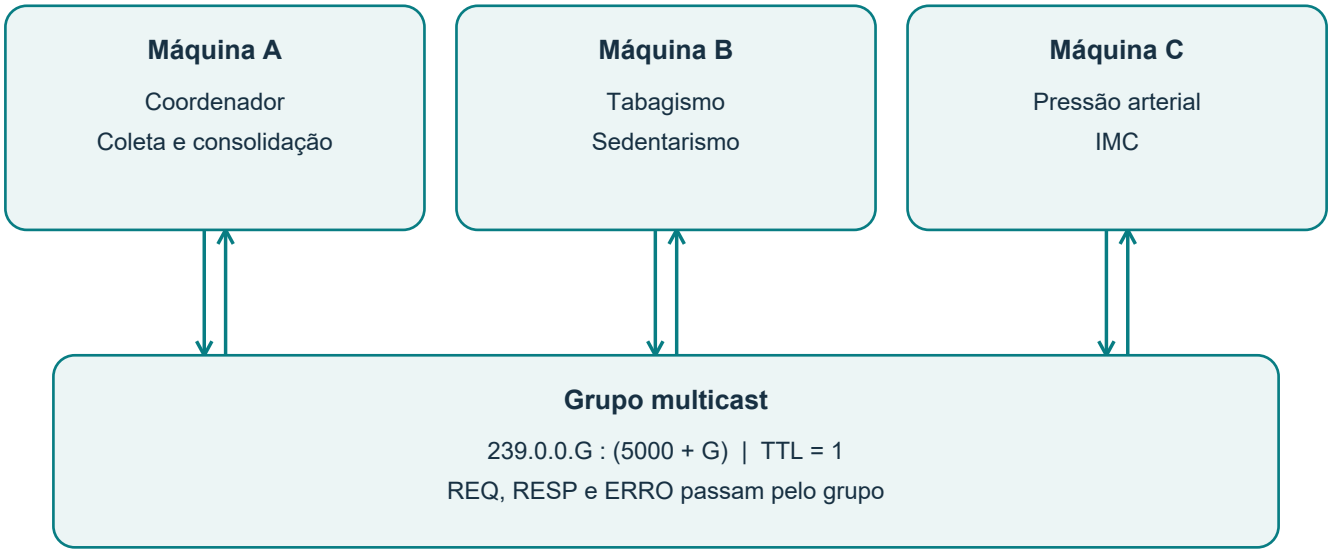
Limite da evidência: a execução registrada ocorreu em uma única máquina. A demonstração presencial em três máquinas, as capturas de tela do laboratório e a identificação dos integrantes ainda devem ser acrescentadas pelo grupo. Nenhum desses dados foi simulado como se tivesse ocorrido.

Identificação: grupo real, integrantes, matrículas, turma e professor: a preencher. O número 7 nas instruções é ilustrativo; deve ser substituído pelo G atribuído ao grupo.

Uso exclusivamente didático. As escalas simplificadas e os estados deste exercício não constituem um instrumento clínico e não devem ser aplicados a pessoas reais.

1. Visão geral e implantação

O projeto separa o processo de entrada e consolidação dos processos de pontuação. Coordenador, Tabagismo, Sedentarismo, Pressao e Imc possuem métodos main próprios. Cada agente examina somente o seu fator. O coordenador encaminha os dados brutos, sem calcular os questionários.



Proposta de implantação. Cada agente é uma JVM independente.

O desenho representa a implantação proposta para o laboratório: máquina A com o coordenador, B com dois agentes comportamentais e C com dois agentes clínicos. As setas representam participação no mesmo grupo; não representam conexões diretas entre processos. Essa distribuição não foi comprovada pelo teste local.

Implantação executada: uma máquina Windows, JVMs independentes. O grupo de testes é 239.0.0.247:5247, distinto do exemplo de grupo 7. A interface usada e a versão de Java estão registradas no resumo da execução. O bytecode foi compilado com --release 17, em um JDK 25.0.1 disponível neste ambiente; não houve execução em um runtime 17 separado.

Componentes compartilhados: Config interpreta parâmetros; Message valida o envelope; MulticastChannel gerencia o socket; EventLog serializa os registros. Evidence concentra as escalas e só é invocada pelos agentes na aplicação. Lpa concentra os nós. PendingEvaluation protege a coleta do coordenador.

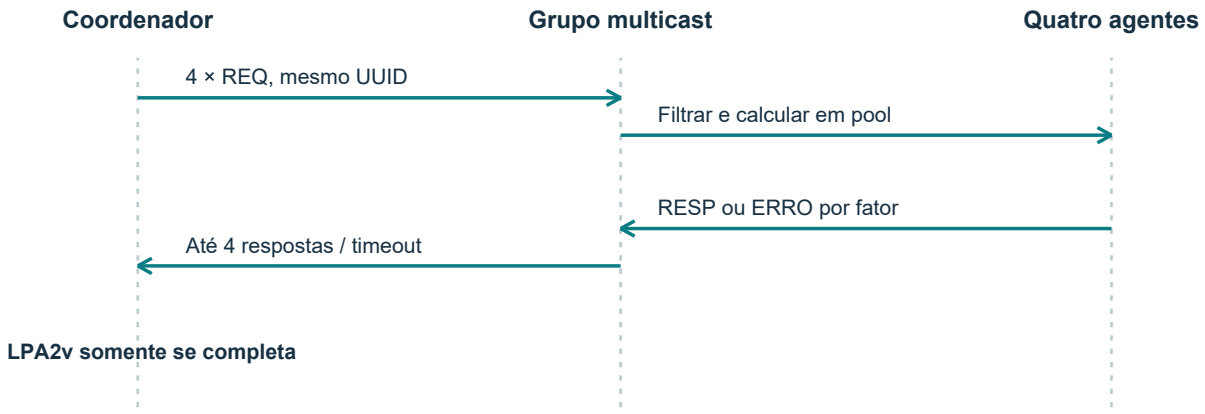
2. Protocolo e sequência

Cada datagrama UTF-8 contém cinco campos separados por ponto e vírgula. A carga usa vírgulas para separar respostas ou medidas, e ponto decimal nos reais. Campos vazios, quebras de linha, delimitadores adicionais e mensagens maiores que 4096 bytes são rejeitados.

Campo	Conteúdo e finalidade
Tipo	REQ, RESP ou ERRO
Avaliação	UUID novo por avaliação; evita misturar pacientes e execuções
Origem	COORD-UUID da instância ou nome do fator
Destino	Nome do fator ou identidade exata do coordenador
Dados	Respostas brutas, evidência numérica ou descrição do erro

```
REQ;avaliacao;COORD-instancia;PRESSAO;150,95
RESP;avaliacao;PRESSAO;COORD-instancia;0.5
ERRO;avaliacao;IMC;COORD-instancia;altura invalida
```

As identidades acima são abreviadas apenas para leitura. A aplicação usa UUIDs completos. Tanto REQ como RESP e ERRO são enviados ao endpoint do grupo, nunca ao IP individual do destinatário.



Os quatro agentes recebem as quatro requisições, mas cada um processa apenas seu destino. O coordenador recebe também suas próprias requisições e as descarta. Em falha, o prazo total termina com os fatores ausentes identificados. Uma resposta ERRO conta como resposta terminal daquele fator, mas impede a consolidação.

Precisão: Double.toString conserva o double no protocolo; Locale.US e três casas são usados somente na apresentação. O formato sugerido no enunciado foi ampliado em precisão para atender à exigência de não arredondar cálculos internos.

3. Ingresso, recepção e encerramento

O endereço padrão é 239.0.0.G e a porta é 5000+G. Ambos podem ser alterados por CLI ou arquivo, sem recompilar. A interface é obrigatória por nome Java ou IPv4 local; a configuração não depende da escolha implícita do sistema operacional.

src/br/edu/cardio/MulticastChannel.java:22-28

```
socket.setReuseAddress(true);
socket.bind(new InetSocketAddress(config.port()));
socket.setNetworkInterface(nic);
socket.setTimeToLive(1);
socket.setSoTimeout(200);
socket.joinGroup(endpoint, nic);
log.event("JOIN", endpoint + " interface=" + nic.getName() + " TTL=1");
```

SO_REUSEADDR é configurado antes do bind para permitir vários processos na mesma porta. A interface serve tanto ao ingresso como ao envio. O TTL fixo em 1 limita saltos IP e satisfaz o alcance exigido; ele não substitui segmentação de rede nem controle de acesso.

src/br/edu/cardio/MulticastChannel.java:54-58

```
@Override public synchronized void close() {
    if (!closed.compareAndSet(false, true)) return;
    try { socket.leaveGroup(endpoint, nic); log.event("LEAVE", endpoint.toString()); }
    catch (IOException e) { log.event("FALHA_LEAVE", e.toString()); }
    finally { socket.close(); log.event("CLOSE", "socket fechado"); }
```

O método close é idempotente por AtomicBoolean. O comando sair e o término normal percorrem esse método; um shutdown hook também o chama em encerramentos normais da JVM. A saída do grupo ocorre antes do fechamento do socket, inclusive quando leaveGroup falha. Um encerramento forçado pelo sistema pode impedir a execução de hooks.

O receive usa timeout de 200 ms para verificar o ciclo de vida; esse valor não é o prazo da avaliação. Cada datagrama recebido é registrado antes da interpretação. O tamanho de buffer excede o limite aceito, permitindo detectar e rejeitar datagramas grandes mesmo quando truncados pelo buffer.

Rastreabilidade: R1, R2, R3 e R10 ficam em Config e MulticastChannel. Os logs reais do teste incluem JOIN, LEAVE e CLOSE. O envio usa sempre o endpoint multicast construído na inicialização.

4. Filtragem, threads e estado compartilhado

src/br/edu/cardio/Agent.java:33-37

```
if (message.origin().equals(factor.name()) || message.type() != Message.Type.REQ
        || !message.destination().equals(factor.name())
        || !message.origin().startsWith("COORD-")) {
    log.event("DESCARTADA", "origem/tipo/destino: " + message.encode());
    continue;
```

O agente exige REQ, destino igual ao próprio fator e origem com identidade de coordenador. Também descarta mensagens originadas pelo seu fator, incluindo as próprias respostas. Esse filtro é roteamento lógico; não autentica o remetente.

src/br/edu/cardio/Agent.java:24-25

```
workers = new ThreadPoolExecutor(config.threads(), config.threads(), 0, TimeUnit.MILLISECONDS,
        new ArrayBlockingQueue<>(128), new ThreadPoolExecutor.AbortPolicy());
```

src/br/edu/cardio/Agent.java:39-42

```
try { workers.execute(() -> process(message)); }
catch (RejectedExecutionException e) {
    log.event("FILA_CHEIA", message.evaluation());
    reply(message, Message.Type.ERRO, "agente ocupado: fila cheia");
```

A recepção submete tarefas ao pool e volta a receber sem executar os cálculos. O padrão é quatro trabalhadores; a fila é limitada a 128 tarefas. Se estiver cheia, o agente registra a sobrecarga e responde com ERRO. Atrasos de processamento, inclusive o atraso artificial de teste, ficam nas threads trabalhadoras.

Evidence é uma coleção de funções sem estado mutável compartilhado. Message, PatientData e os resultados são records imutáveis. EventLog usa métodos synchronized para impedir mistura de linhas; o envio e o fechamento do canal são sincronizados. AtomicBoolean protege o encerramento. Essas operações curtas de rede e log ainda podem levar tempo; a garantia R8 separa o processamento da recepção, não promete recepção em tempo real.

No coordenador, um receptor independente valida origem, destino, tipo e avaliação antes de entregar a resposta a PendingEvaluation. O mapa de avaliações é sincronizado, e cada coleta tem seu próprio monitor. Assim, escrita das respostas e leitura do resultado são protegidas contra corridas.

Teste realizado: dois pedidos IMC, com dois trabalhadores e atraso de 600 ms, iniciaram o processamento antes da primeira resposta. Os registros PROCESSANDO e ENVIADA comprovam esse comportamento no teste local.

5. Correlação, duplicidade e timeout

src/br/edu/cardio/PendingEvaluation.java:20-31

```
public synchronized String accept(Factor factor, Message message) {
    if (values.containsKey(factor) || errors.containsKey(factor)) return "DUPLICADA";
    if (finished || System.nanoTime() >= deadline) return "TARDIA";
    if (message.type() == Message.Type.ERRO) errors.put(factor, message.data());
    else {
        double value;
        try { value = Double.parseDouble(message.data()); }
        catch (NumberFormatException e) { return "EVIDENCIA_INVALIDA"; }
        if (!Double.isFinite(value) || value < 0 || value > 1) return "EVIDENCIA_INVALIDA";
        values.put(factor, value);
    }
    notifyAll();
```

A primeira resposta válida de cada fator prevalece. Valores RESP precisam ser finitos e estar em [0,1]. ERRO guarda a descrição sem inventar evidência. Respostas inválidas não ocupam a posição do fator, permitindo que uma resposta válida ainda chegue no prazo. A duplicata é detectada antes da condição de término para poder registrá-la mesmo depois da consolidação.

src/br/edu/cardio/PendingEvaluation.java:35-41

```
public synchronized Snapshot await() throws InterruptedException {
    while (values.size() + errors.size() < Factor.values().length) {
        long remaining = deadline - System.nanoTime();
        if (remaining <= 0) break;
        TimeUnit.NANOSECONDS.timedWait(this, remaining);
    }
    finished = true;
```

O prazo usa System.nanoTime, um relógio monotônico adequado para intervalos. É criado antes dos quatro envios e não reinicia a cada resposta. timedWait libera o monitor para o receptor, e o laço reavalia a condição após cada notificação. Uma resposta que chega depois do limite é descartada mesmo se o laço de espera ainda não tiver retomado.

A avaliação termina ao receber quatro respostas terminais ou ao vencer o prazo. Somente quatro evidências válidas permitem Lpa.consolidate. O coordenador informa separadamente fatores com erro e fatores sem resposta. Por exemplo, altura zero produz ERRO_AGENTE IMC, enquanto ausência de IMC produz AUSENTE IMC.

O identificador UUID da avaliação e a identidade COORD-UUID do destinatário separam coordenadores no mesmo grupo. O histórico limitado às últimas 256 avaliações permite reconhecer duplicatas recentes sem crescimento ilimitado de memória. Respostas mais antigas são descartadas como avaliação desconhecida.

6. Escalas e validação pelos agentes

Agente	Cálculo obrigatório
Tabagismo	Soma de T1..T5 dividida por 13
Sedentarismo	Soma de S1..S5 dividida por 9
Pressão	Máximo entre clamp((PAS-120)/60) e clamp((PAD-80)/30)
IMC	clamp((peso/(altura²)-25)/15)

clamp(x) = max(0,min(1,x)). PAS, PAD e peso precisam ser positivos e finitos. A altura aceita é de 0,50 a 2,50 m, inclusive, faixa didática explicitada pelo projeto. NaN e infinitos são rejeitados. Os questionários exigem exatamente cinco inteiros, e cada pergunta tem seu conjunto de pontos permitidos.

src/br/edu/cardio/Evidence.java:9-14

```
case TABAGISMO -> questionnaire(data, new int[][]{
        {0,1,2,4}, {0,1,2,3}, {0,1,2,3}, {0,1}, {0,1,2}}, 13);
case SEDENTARISMO -> questionnaire(data, new int[][]{
        {0,1,2,3}, {0,1,2}, {0,1}, {0,1,2}, {0,1}}, 9);
case PRESSAO -> pressure(data);
case IMC -> bmi(data);
```

O conjunto de T1 é {0,1,2,4}; aceitar qualquer inteiro de 0 a 4 seria incorreto. S3 e S5 só aceitam 0 ou 1. A rotina questionnaire valida cada ponto, soma e divide em double, sem arredondar.

src/br/edu/cardio/Evidence.java:33-46

```
private static double pressure(String data) {
    String[] fields = split(data, 2);
    double systolic = positive(fields[0], "pressao sistolica");
    double diastolic = positive(fields[1], "pressao diastolica");
    return Math.max(clamp((systolic - 120) / 60), clamp((diastolic - 80) / 30));
}

private static double bmi(String data) {
    String[] fields = split(data, 2);
    double weight = positive(fields[0], "peso");
    double height = positive(fields[1], "altura");
    if (height < 0.5 || height > 2.5)
        throw new IllegalArgumentException("altura invalida: faixa didatica de 0.50 a 2.50 m");
    return clamp((weight / (height * height) - 25) / 15);
```

As duas interpolações de PA são limitadas separadamente antes da seleção da maior evidência. No IMC, o cálculo usa peso e altura originais, sem arredondar primeiro o IMC para duas casas.

Limitações das escalas: baixo peso cai em evidência zero no agente IMC, apesar de não equivaler a ausência de problemas de saúde. Não há avaliação de idade, sexo, histórico familiar, medicamentos, sintomas ou interações entre fatores. O resultado representa exclusivamente a proposição didática definida no exercício.

7. Rede LPA2v e classificação

As fórmulas que não vieram nas imagens do texto colado foram reconstruídas pelo exemplo resolvido e pelas descrições. Para cada nó, a segunda evidência favorável b é convertida em desfavorável por λ = 1-b.

src/br/edu/cardio/Lpa.java:10-17

```
public static Node node(double first, double second) {
    for (double value : new double[]{first, second})
        if (!Double.isFinite(value) || value < 0 || value > 1)
            throw new IllegalArgumentException("evidencia fora de [0,1]");
    double lambda = 1 - second;
    double gc = first - lambda;
    double gct = first + lambda - 1;
    return new Node(first, lambda, gc, gct, (gc + 1) / 2);
```

NAP1 = nó(μT, μS); NAP2 = nó(μPA, μIMC); NAP3 = nó(μE1, μE2). O coordenador exibe μ, λ, Gc, Gct e μE de cada nó. Somente o NAP3 determina o estado final, obedecendo à prioridade exata da tabela.

Condição, testada nesta ordem	Estado	Classificação
Gc ≥ 0,5	V	Risco elevado
Gc ≤ -0,5	F	Risco baixo
Gct ≥ 0,5	T	Inconclusivo
Gct ≤ -0,5	Paracompleto (⊥)	Inconclusivo
Demais valores	Não extremo	Indefinido, com tendência pelo sinal de Gc

Se Gc=0 no estado não extremo, a saída diz sem tendência. A representação textual PARACOMPLETO nos terminais evita dependência do símbolo ⊥; ela corresponde ao estado exigido. Os testes verificam os limites inclusive e a prioridade de Gc quando certeza e contradição atingem 0,5 ao mesmo tempo.

O que a LPA acrescenta à média? Substituindo λ=1-b: Gc=a-(1-b)=a+b-1. Logo μE=(Gc+1)/2=(a+b)/2. Além da média, o nó preserva Gct=a-b, que explicita a divergência orientada entre entradas. A saída escalar isolada perde essa informação. No NAP3, μE é a média dos quatro fatores, mas Gct distingue os casos 4 e 5, ambos com média 0,5.

Com essa regra, Gct negativo pode surgir de duas evidências favoráveis completas e divergentes. Por isso o rótulo paracompleto deve ser interpretado conforme a rede didática, sem afirmar que faltaram respostas.

8. Resultados dos seis casos

Os valores abaixo foram extraídos da saída do coordenador no teste multicast real, com JVMs independentes na mesma máquina. Cada célula mostra esperado / obtido. Os cálculos unitários independentes da rede também verificam as mesmas escalas e a tolerância de 0,001.

Caso	μT E / O	μS E / O	μPA E / O	μIMC E / O
1	0.000 / 0.000	0.000 / 0.000	0.000 / 0.000	0.000 / 0.000
2	1.000 / 1.000	1.000 / 1.000	1.000 / 1.000	1.000 / 1.000
3	0.615 / 0.615	0.778 / 0.778	0.500 / 0.500	0.336 / 0.336
4	1.000 / 1.000	1.000 / 1.000	0.000 / 0.000	0.000 / 0.000
5	0.000 / 0.000	0.000 / 0.000	1.000 / 1.000	1.000 / 1.000
6	0.308 / 0.308	0.333 / 0.333	0.250 / 0.250	0.179 / 0.179

Caso	Gc E / O	Gct E / O	Estado obtido
1	-1.000 / -1.000	0.000 / 0.000	F
2	1.000 / 1.000	0.000 / 0.000	V
3	0.115 / 0.115	0.279 / 0.279	Não extremo
4	0.000 / 0.000	1.000 / 1.000	T
5	0.000 / 0.000	-1.000 / -1.000	Paracompleto
6	-0.465 / -0.465	0.106 / 0.106	Não extremo

Os casos 1 e 2 atingem os extremos de risco baixo e elevado. O caso 3 reproduz o exemplo: Gc=0,115, Gct=0,279 e μE=0,557, estado não extremo com tendência a elevado. O caso 6 mantém tendência a baixo, pois Gc=-0,465 ainda não alcança -0,5.

Caso 4: ambos os fatores comportamentais estão no máximo e os clínicos no mínimo. NAP1 produz 1 e NAP2 produz 0. O nó final recebe (μ,λ)=(1,1), Gc=0 e Gct=1: inconsistente T.

Caso 5: o comportamento está no mínimo e os fatores clínicos no máximo. NAP1 produz 0 e NAP2 produz 1. O nó final recebe (μ,λ)=(0,0), Gc=0 e Gct=-1: paracompleto.

A classificação inconclusivo segue o enunciado e evita ocultar o desacordo entre blocos. Contudo, os dois casos têm quatro respostas válidas; no caso 5 não houve perda de informação por timeout. A orientação do complemento explica a assimetria T/⊥. A interpretação não valida os rótulos como decisão clínica.

Precisão: cálculos em double até a saída; os valores observados e esperados podem diferir na terceira casa se alguém refizer contas com evidências já arredondadas. O arquivo calculos.csv preserva nove casas para inspeção, e os datagramas preservam a representação completa de double.

9. Falhas, concorrência e registros reais

Java=25.0.1 OS=Windows 11 interface=ethernet_32773 IPv4=239.0.0.247:5247

Cenário	Resultado observado
Seis casos normais	6/6 aprovados; valores dentro de 0,001
Agente ausente	IMC não respondeu; timeout de 3000 ms; incompleta
Reingresso	IMC reiniciou; caso 3 voltou a consolidar
Dois coordenadores	Casos 1 e 2 separados por UUID e destino
Tabagismo duplicado	Segunda resposta descartada e registrada
Altura zero	ERRO do IMC; sem consolidação
Pool e saída limpa	Tarefas concorrentes; sair gerou LEAVE/CLOSE

O harness IntegrationTests inicia JVMs pelo ProcessBuilder, escreve comandos em stdin e verifica saídas e logs. Essa comunicação de controle pertence ao teste; a comunicação da aplicação entre coordenadores e agentes permanece integralmente multicast.

```
2026-10-19T14:28:40.312076600-03:00 [coordenador] AUSENTE IMC nao respondeu no prazo
2026-10-19T14:28:42.983728300-03:00 [coordenador] DESCARTADA_DUPLICADA
    RESP;<uuid>;TABAGISMO;COORD-<uuid>;0.6153846153846154
2026-10-19T14:28:43.707136800-03:00 [coordenador] ERRO_AGENTE IMC: altura deve ser finita e positiva
```

Os excertos acima vêm dos arquivos reais da execução e foram abreviados para caber na página. São transcrições textuais, não capturas de tela. Os registros integrais contêm data, hora com fuso, processo, thread e mensagem enviada, recebida ou descartada.

O teste local cobre os seis cenários do roteiro e acrescenta a verificação do pool com processamento atrasado. Não comprova transporte entre máquinas, comportamento de switches nem configuração do firewall do laboratório. Também não injeta perda aleatória de datagramas: a ausência é provocada pelo encerramento do agente.

10. Caso 6 refeito manualmente

Tabagismo: pontos 2+1+1+0+0=4; μT=4/13=0,3076923077. Sedentarismo: 1+1+0+1+0=3; μS=3/9=0,3333333333.

Pressão: PAS 135 e PAD 85. Evidências (135-120)/60=0,25 e (85-80)/30=0,1666666667. O máximo é μPA=0,25.

IMC=80/(1,70²)=27,6816608997. μIMC=(27,6816608997-25)/15=0,1787773933.

Nó	μ	λ	Gc	Gct	μE
NAP1	0,307692	0,666667	-0,358974	-0,025641	0,320513
NAP2	0,250000	0,821223	-0,571223	0,071223	0,214389
NAP3	0,320513	0,785611	-0,465098	0,106124	0,267451

NAP1: λ1=1-μS=0,6666666667; Gc1=μT-λ1=-0,3589743590; Gct1=μT+λ1-1=-0,0256410256; μE1=(Gc1+1)/2=0,3205128205.

NAP2: λ2=1-μIMC=0,8212226067; Gc2=0,25-λ2=-0,5712226067; Gct2=0,25+λ2-1=0,0712226067; μE2=0,2143886967.

NAP3: μ3=μE1=0,3205128205; λ3=1-μE2=0,7856113033. Gc3=-0,4650984828; Gct3=0,1061241239; μE3=0,2674507586.

Como -0,5 < Gc3 < 0,5 e -0,5 < Gct3 < 0,5, o estado é não extremo. O sinal negativo de Gc3 produz risco indefinido com tendência a baixo. Nenhum arredondamento foi aplicado entre os nós; a tabela apresenta somente seis casas para leitura.

11. Questões de comunicação e decisões

Por que multicast? Participantes ingressam em um grupo lógico, sem manter no coordenador uma lista de IPs. Broadcast atinge indiscriminadamente o domínio de broadcast; multicast é direcionado aos membros, condicionado à infraestrutura de rede. Como o protocolo atual mantém uma REQ por fator, não se afirma que os quatro pedidos virem um único datagrama. A vantagem principal aqui é endereçamento do grupo e adesão dinâmica; a rede ainda pode replicar tráfego de forma ampla se não fizer filtragem multicast.

E um quinto agente? O canal multicast permanece igual, mas o sistema não é extensível automaticamente: seria necessário acrescentar um Factor, a coleta de entrada, seu processo e sua validação, a espera por cinco resultados e uma nova topologia LPA2v. A rede atual agrupa quatro fatores em uma árvore balanceada. Inserir um quinto sem definir os pesos alteraria a interpretação da média.

Que falhas UDP são detectadas? Timeout acusa fator sem resposta, mas não distingue perda de REQ, perda de RESP, lentidão ou queda do agente. O sistema detecta formato inválido, evidência inválida, duplicação, origem/destino impróprios, avaliação desconhecida e resposta tardia. Ordem de chegada não importa porque há correlação por UUID e fator. Sem retransmissão, uma única perda pode tornar a avaliação incompleta.

Que falhas passariam despercebidas? Um agente com cálculo errado que responda com valor em [0,1], dados falsificados com identidades conhecidas, recepção parcial de uma mensagem válida do ponto de vista sintático e versões semanticamente incompatíveis podem não ser identificados. O protocolo não usa assinatura, autenticação nem versão. O primeiro resultado válido de uma instância duplicada prevalece mesmo que as instâncias tenham implementações divergentes.

TTL maior que 1 e grupos iguais: se houver roteamento multicast entre redes, mensagens podem alcançar outro laboratório, ampliar tráfego e fazer agentes de lá responderem a solicitações daqui. UUIDs reduzem a mistura entre avaliações, mas não impedem que um agente remoto com o mesmo nome processe a mesma REQ. Duplicatas ou resultados de uma versão diferente podem interferir. TTL 1, grupo/porta por turma e configuração correta reduzem esse problema, sem oferecer autenticação.

Alternativas descartadas: TCP, RMI e unicast descumpririam a tarefa. Frameworks de agentes e dependências externas foram evitados por Java SE puro. Serialização nativa de objetos foi substituída por texto pequeno e inspecionável. Thread por requisição foi preterida por pool e fila limitada para conter consumo de recursos. Retransmissão e cache de idempotência exigiriam um protocolo adicional e não são parte da versão mínima.

12. Comparação, pendências e entrega

Comparação com MultiagenteCambial: seu código não foi fornecido, portanto não é possível atribuir características concretas a essa implementação. Se os agentes do exemplo cambial compartilhavam uma única JVM, a separação em processos aqui oferece isolamento de falhas, execução em máquinas diferentes e reingresso sem reiniciar os outros agentes. Em contrapartida, acrescenta serialização, perda de mensagens, timeout, correlação, logs distribuídos, maior consumo de memória e operação de rede. Essa comparação é condicional e deve ser ajustada pelo grupo após consultar o exemplo da disciplina.

Rastreabilidade dos requisitos: R1 a R3: Config e MulticastChannel. R4: Message. R5: filtros de Agent e Coordenador. R6 e R7: PendingEvaluation e histórico do coordenador. R8: ThreadPoolExecutor, funções sem estado e monitores. R9: EventLog e registros em cada caminho de recepção, envio e descarte. R10: close idempotente, comando sair e shutdown hooks.

Execução pelo avaliador: compilar com build.ps1, consultar --listar-interfaces e iniciar os cinco processos com --interface e --grupo iguais quanto ao grupo. A interface é específica de cada máquina. Sem PowerShell, usar os comandos javac/java descritos no README. A aplicação e os testes não precisam de Maven, Gradle, Python ou conexão à internet.

Pendências antes da entrega presencial: preencher identificação; executar o roteiro em três máquinas; coletar logs distribuídos e capturas reais; registrar IP/interface e a distribuição efetiva; incorporar os resultados ao relatório. Se houver bloqueio multicast, registrar a comunicação ao professor e a alternativa de demonstração autorizada.

Integrante	Responsabilidade efetivamente realizada
A preencher	Protocolo e ciclo de vida do multicast
A preencher	Agentes, validação e escalas
A preencher	Coordenador, correlação e LPA2v
A preencher	Testes, operação e documentação

A tabela é uma sugestão de organização a ser substituída pela divisão real. Não declara participação de pessoas não identificadas. Todos devem estar aptos a explicar qualquer processo.

Fontes e arquivos de apoio: enunciado fornecido pelo usuário; código Java em src; testes em test; README.md; docs/ROTEIRO-LABORATORIO.md; evidencias/local/calculos.csv; logs da execução apontada por evidencias/local/ultimo-teste.txt. Não foram consultadas fontes clínicas externas; as fórmulas e os resultados esperados seguem a tarefa.